



Technical Test – Full-Stack developer (Backend oriented)

Dear candidate,

As part of the recruitment process at Bagira Systems, you are required to complete this assignment. You will receive the assignment at the time of your choice (e.g., 9 AM) and must submit your solution within 24 hours (e.g., by 9 AM the next day) ***please read the description carefully before starting.***

Objectives:

Implement missing parts in a provided code skeleton (backend and frontend) for a small web application used to create training scenarios and manage entities inside each scenario

Instructions:

1. Clone this repository – [Bagira-Assignment](#) and create a fork with the following structure – ‘BagiraAssignment_<YourName>’.
2. Make sure to work on the folder named ‘**Backend-Oriented**’.
3. The Repo contains frontend skeleton for React and Angular implementation – choose one to implement and specify which one you chose.
4. Follow the **README** instructions on the backend and frontend folders for detailed instructions.
5. Implement the missing functionality marked in the code (TODOs) and ensure the app works end-to-end.

Requirements:

1. Data rules:
 - EntityType must be one of the following - Soldier, Tank, Drone, Aircraft, Vehicle, Civilian.
 - TaskForce must be one of the following - Friendly, Enemy.
 - Latitude must be between -90 and 90.
 - Longitude must be between -180 and 180.
 - Entities always belong to exactly one Scenario (no orphan entities).
2. Screens:
 - Scenario List – view all scenarios, navigate to details, create.
 - Create Scenario – the form should contain name and description.
 - Scenario Details – view scenario info and manage its entities.
 - Create Entity – the form must include:
 - EntityType
 - TaskForce
 - Name/Callsign
 - Latitude
 - Longitude
3. Backend Features:
 - Data Access Layer:
 - Implement ScenarioRepository, EntityRepository with chosen data storage (In-Memory or Database).
 - If using database: set up DbContext, migrations, and connection strings.
 - API Endpoints – implement all endpoints on ScenariosController, EntitiesController and SearchController.
 - Filtering & Sorting - server-side implementation
 - Search:
 - Global search endpoint that searches across scenarios and entities.
 - Search scenarios by name and description.
 - Search entities by name/callsign.
 - Error Handling:
 - Proper HTTP status codes.
 - ErrorResponse DTO for validation errors.
 - Validation – data validation based on the data rules.
4. Frontend:
 - Connect frontend services to backend APIs.
 - Implement form submission and data fetching logic.
 - Implement error handling and loading states.



- Ensure validation feedback is displayed to users.
 - Complete remaining TODOs in frontend code.
5. Docker:
- After implementing the core functionality, containerize the solution using Docker:
 - i. Create Docker files for Backend and Frontend
 - ii. Create a docker-compose.yml.
 - iii. Ensure the solution runs with: docker-compose up
 - iv. Update the main README with Docker instructions
 - Notes:
 - i. Use multi-stage builds for frontend (build + serve)
 - ii. Configure CORS in backend to allow frontend origin
 - iii. Use environment variables for API base URL in frontend
 - iv. Document any required environment variables
6. Data persistency:
- **Bonus:** If you have time, connect to a DB of your choice
7. Optional bonus:
- Add tests for validation (backend) or key UI flows (frontend).
 - Implement Update and Delete capabilities end-to-end for Scenarios and Entities controller.
 - Improve UX polish.

Acceptance criteria:

- User can create a scenario and see it in the scenario list.
- User can open scenario details and view entities belonging to that scenario only.
- User can select TaskForce (Friendly/Enemy) when creating entities, and TaskForce is displayed in entity lists.
- User can filter scenarios by name/description (server-side filtering works).
- User can filter entities by Type and TaskForce (server-side filtering works).
- User can sort scenarios and entities in various fields (server-side sorting works).
- User can search for scenarios and entities using the global search endpoint.
- Backend validates all input fields (coordinates, types, required fields) and returns appropriate error responses.
- Backend returns proper HTTP status codes for all operations.
- Data persistence works correctly (in-memory or database).
- Solution runs locally.
- Solution runs in Docker using docker-compose up with all services properly configured and communicating.
- Using clean architecture.
- Code quality and maintainability.
- Validation and error handling are properly implemented.
- Reasonable scope management within the timebox.

Please provide:

- A link to your branch.
- Short notes (bullet points) describing what you completed and any trade-offs.

Good Luck!